

Maîtriser et optimiser

Requêtes SQL avancées

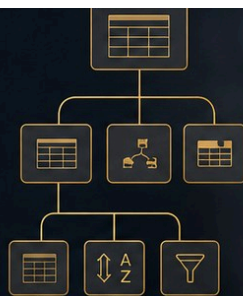
Pour des applications performantes
et évolutives



Plan



Mesurer



1	
2	
3	
4	
5	

SQL - EXPERT

SQL

Fenêtres, CTE, EXPLAIN, index et
intégrité pour des requêtes de production.

DanielCraft - Pack Data/SQL

Sommaire

1. Après l'intermédiaire : SQL expert
 - Ce que tu vas apprendre
 - A retenir
2. Fonctions fenetre en profondeur
 - Familles utiles
 - PARTITION BY + ORDER BY
 - A retenir
3. CTE : WITH pour clarifier
 - Pourquoi s'en servir
 - A retenir
4. CTE recursive : hierarchies
 - Garde-fous
 - A retenir
5. EXPLAIN : lire le plan
 - Signaux a surveiller
 - A retenir
6. Index : strategie avantee
 - Types d'idees
 - Anti-patterns
 - A retenir
7. Integrite : contraintes et cles
 - Outils
 - A retenir
8. Transactions et niveaux d'isolement
 - Niveaux (idee)
 - Verrous
 - A retenir
9. Partitionnement (idee)
 - Interet
 - Exemple mental
 - A retenir
10. Vues materialisees
 - Quand
 - Contrepartie
 - A retenir
11. JSON dans SQL (idee)
 - Usages
 - Prudence
 - A retenir
12. Patterns de performance

- [Checklist rapide](#)
- [Pagination](#)
- [A retenir](#)

13. [Qualite des requetes](#)

- [Style expert](#)
- [Anti-patterns](#)
- [A retenir](#)

14. [Mini-projet : tableau de bord SQL](#)

- [Livrables](#)
- [Jeu de donnees](#)
- [A retenir](#)

15. [Ce qu'il faut retenir](#)

- [Carte expert SQL](#)
- [Mindset](#)
- [A retenir](#)

16. [Atelier : fenetres](#)

- [Mission](#)
- [Criteres](#)
- [A retenir](#)

17. [Atelier : EXPLAIN + index](#)

- [Mission](#)
- [Questions](#)
- [A retenir](#)

18. [Erreurs classiques niveau expert](#)

- [A retenir](#)

19. [Bonnes pratiques SQL expert](#)

- [Checklist prod](#)
- [A retenir](#)

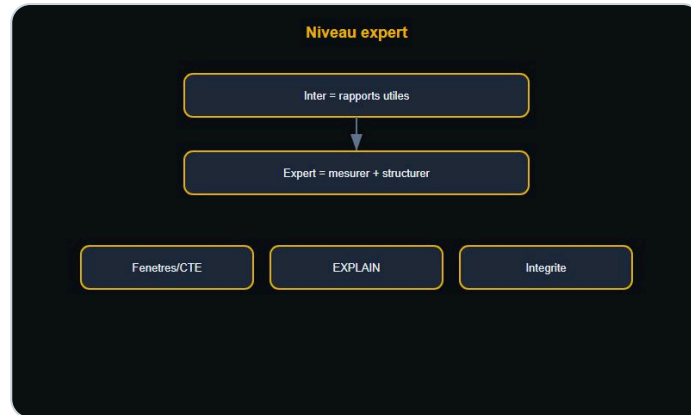
20. [Quiz SQL expert](#)

- [Bareme](#)
- [A retenir](#)

21. [Bravo !](#)

- [Tu sais maintenant](#)
- [Et maintenant ?](#)
- [A retenir](#)

Après l'intermédiaire : SQL expert



Du rapport utile à la maîtrise moteur.



Couverture SQL Expert.

Tu sais écrire des sous-requêtes, filtrer des dates, utiliser HAVING. Le niveau **expert**, c'est **expliquer**, **mesurer** et **structurer** : fenêtres, CTE, plans d'exécution, index utiles, transactions solides.

> **Astuce DanielCraft** - Expert SQL = lisibilité + perf + intégrité. Pas de magie noire.

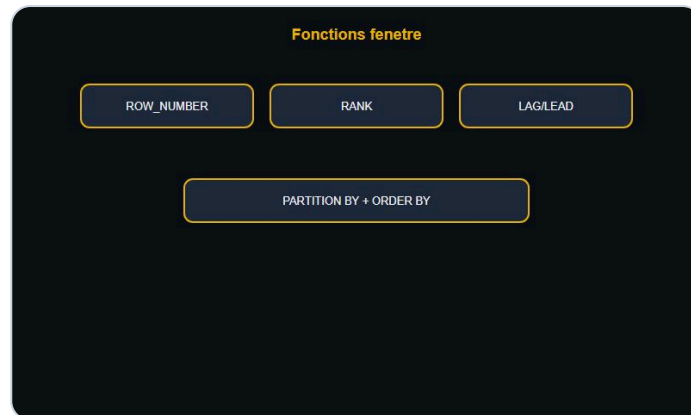
Ce que tu vas apprendre

- Fenêtres (ROW_NUMBER, RANK, LAG/LEAD).
- CTE et CTE récursives.
- EXPLAIN, index, verrous.
- Intégrité, partitions, vues matérialisées.

A retenir

- Expert = comprendre ce que le moteur fait.
- Une requête claire bat une requête "astucieuse" opaque.

Fonctions fenetre en profondeur



Fenêtres sans écraser les lignes.

Une **fenêtre** calcule sur un groupe de lignes **sans les écraser** (contrairement à GROUP BY).

```
SELECT client_id, montant,
       ROW_NUMBER() OVER (PARTITION BY client_id ORDER BY montant DESC) AS rang
FROM commandes;
```

Familles utiles

Fonction	Role
ROW_NUMBER	Rang unique 1, 2, 3...
RANK / DENSE_RANK	Rangs avec egalites
SUM/AVG OVER	Totaux courants
LAG / LEAD	Ligne precedente / suivante

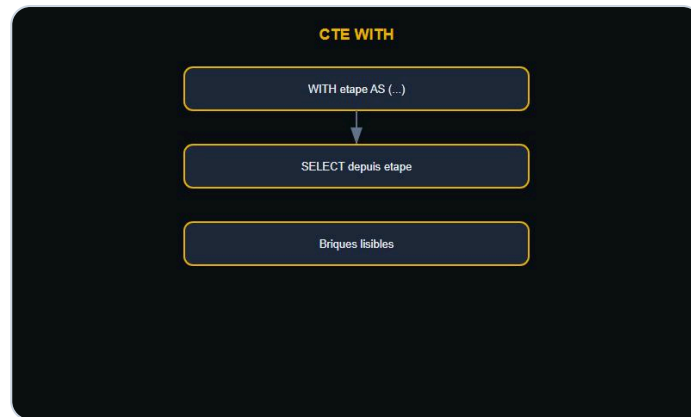
PARTITION BY + ORDER BY

- **PARTITION BY** = découper les groupes.
- **ORDER BY** dans OVER = ordre **dans** la fenetre.

A retenir

- Fenetre = detail conserve + calcul de voisinage.
- Commence par ROW_NUMBER avant les cas complexes.

CTE : WITH pour clarifier



WITH pour clarifier.

Une **CTE** (Common Table Expression) nomme un resultat intermediaire avec **WITH**.

```
WITH gros AS (  
  SELECT client_id, SUM(montant) AS ca  
  FROM commandes  
  GROUP BY client_id  
  HAVING SUM(montant) > 1000  
)  
SELECT c.prenom, g.ca  
FROM gros g  
JOIN clients c ON c.id = g.client_id;
```

Pourquoi s'en servir

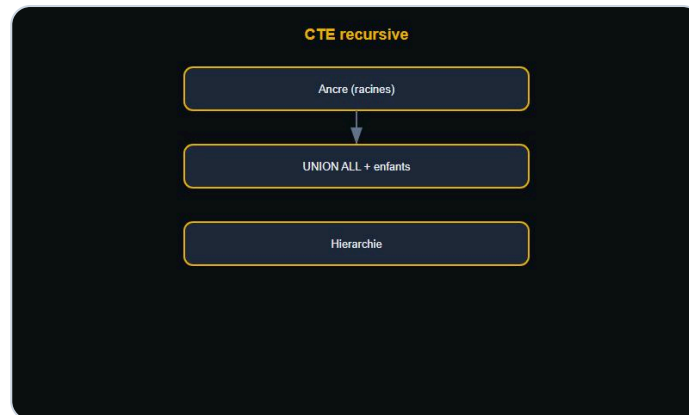
- Lire la requete comme des etapes.
- Reutiliser le meme bloc plusieurs fois.
- Preparer une recursive (chapitre suivant).

> **Piege** - Une CTE n'est pas toujours "materialisee" : le moteur peut l'inliner. Teste avec EXPLAIN.

A retenir

- WITH = briques nommees, plus claires qu'une sous-requete geante.

CTE recursive : hierarchies



Hierarchies en recursive.

Une CTE **recursive** se reference elle-meme : ideal pour arbres (categories, org chart).

```
WITH RECURSIVE arbre AS (  
  SELECT id, parent_id, nom, 1 AS niveau  
  FROM categories WHERE parent_id IS NULL  
  UNION ALL  
  SELECT c.id, c.parent_id, c.nom, a.niveau + 1  
  FROM categories c  
  JOIN arbre a ON c.parent_id = a.id  
)  
SELECT * FROM arbre;
```

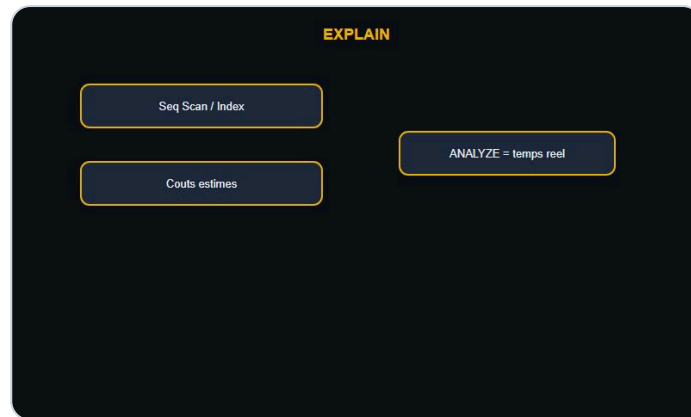
Garde-fous

- Toujours une ancre (partie non recursive).
- Limiter la profondeur si besoin.
- Eviter les cycles (ou les detecter).

A retenir

- Recursive = ancre + UNION ALL + auto-jointure.
- Parfait pour hierarchie, pas pour tout.

EXPLAIN : lire le plan



Lire le plan.

EXPLAIN montre comment le moteur execute ta requete (scan, index, jointure...).

```
EXPLAIN ANALYZE
SELECT * FROM commandes WHERE client_id = 42;
```

Signaux a surveiller

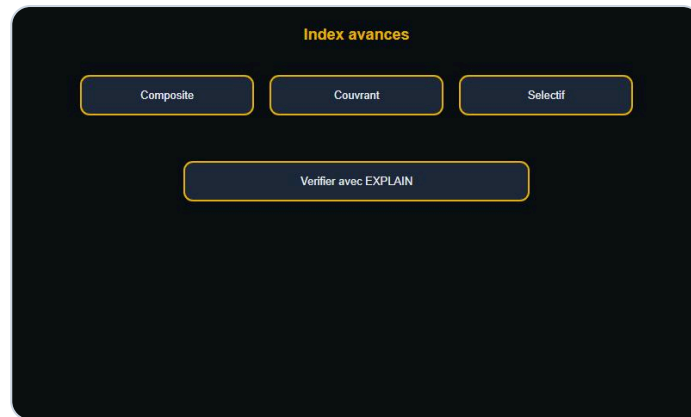
- **Seq Scan** sur grosse table + filtre selectif → index manquant ?
- **Nested Loop** vs **Hash Join** : selon volumes.
- Cout estime vs temps reel (ANALYZE).

> **Astuce DanielCraft** - EXPLAIN avant d'ajouter un index "au feeling".

A retenir

- Le plan > l'intuition.
- Mesure sur des volumes realistes.

Index : strategie avantee



Index cibles.

Un index accelere certaines lectures, ralentit un peu les ecritures.

Types d'idees

- Index sur colonnes de **WHERE** / **JOIN** frequents.
- Index **composite** : ordre des colonnes compte ((a, b) aide `WHERE a = ?` puis `a = ? AND b = ?`).
- Index **couvrant** : toutes les colonnes lues sont dans l'index (moins de retours table).

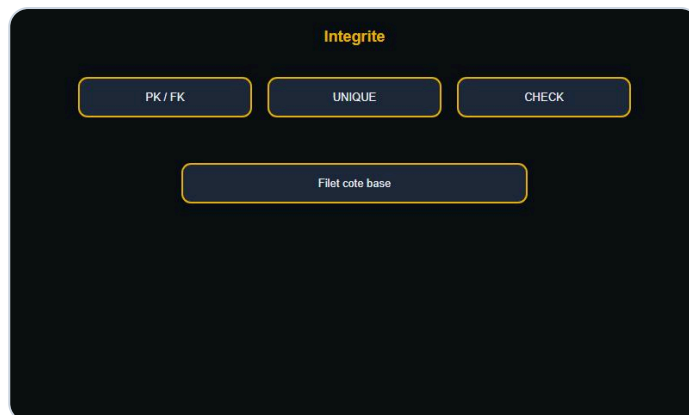
Anti-patterns

- Indexer toutes les colonnes.
- Index inutilises (maintenance inutile).
- Fonctions sur colonnes indexees (`WHERE LOWER(email) = ...`) qui empechent l'usage.

A retenir

- Index = outil cible, pas decoration.
- Verifie avec EXPLAIN apres creation.

Integrite : contraintes et cles



Contraintes en base.

La base protege les donnees **meme si l'app se trompe**.

Outils

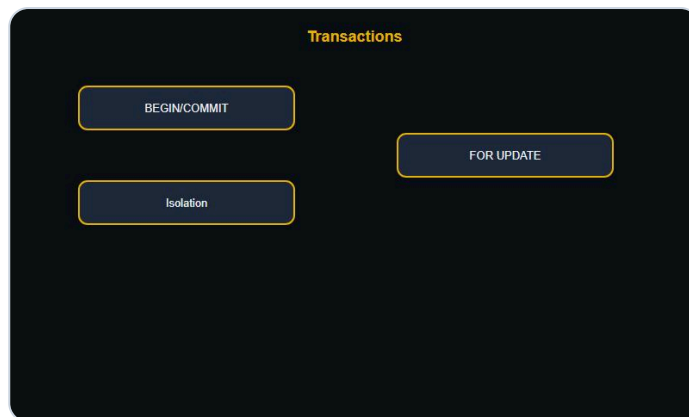
Contrainte	Role
PRIMARY KEY	Identite unique
FOREIGN KEY	Lien valide vers une autre table
UNIQUE	Pas de doublon metier
CHECK	Regle simple (montant > 0)
NOT NULL	Champ obligatoire

```
ALTER TABLE commandes
  ADD CONSTRAINT fk_client
  FOREIGN KEY (client_id) REFERENCES clients(id);
```

A retenir

- Integrite en base = filet de securite.
- Soft delete et FK : predire le comportement (CASCADE ou non).

Transactions et niveaux d'isolement



Tout ou rien + isolation.

Une **transaction** = tout ou rien (BEGIN / COMMIT / ROLLBACK).

Niveaux (idée)

Niveau	Idee
READ COMMITTED	Voir seulement les commits (souvent default)
REPEATABLE READ	Snapshot plus stable
SERIALIZABLE	Plus strict, plus de conflits possibles

Verrous

- Deux ventes sur le meme stock : attention race condition.
- `SELECT ... FOR UPDATE` pour verrouiller une ligne avant update.

> **Piege** - Transaction longue = verrous longs = apps qui attendent.

A retenir

- Isolation = compromis coherence / concurrency.
- Court et clair > long et flou.

Partitionnement (idee)



Decouper pour lire moins.

Partitionner = decouper une grosse table en morceaux (par date, region...).

Interet

- Scanner seulement la partition utile (prune).
- Archiver / detacher d'anciennes partitions.

Exemple mental

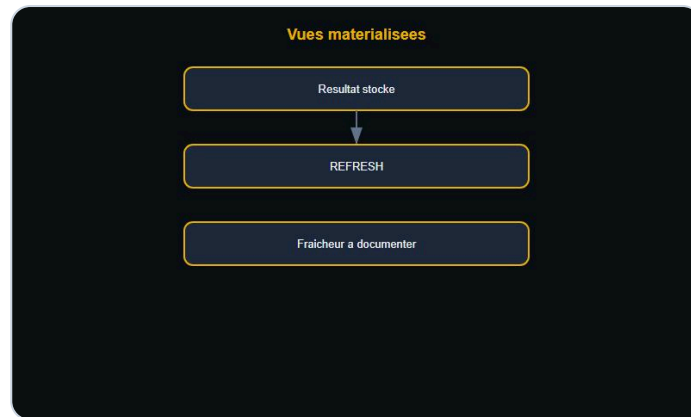
commandes partitionnee par mois : `WHERE date >= '2026-01-01' AND date < '2026-02-01'` ne lit que janvier.

> **Astuce DanielCraft** - Partitionne quand la table est **vraiment** grosse, pas par snobisme.

A retenir

- Partition = organisation physique, pas magie SQL.
- La cle de partition doit matcher tes filtres.

Vues materialisees



Cache SQL rafraichi.

Une **vue** calcule a la volee. Une **vue materialisee** stocke le resultat (a rafraichir).

Quand

- Rapports lourds lus souvent.
- Agregats stables (refresh nocturne OK).

```
-- Idee (syntaxe selon moteur)
REFRESH MATERIALIZED VIEW rapport_mensuel;
```

Contrepartie

- Donnees potentiellement **stale**.
- Espace disque + cout de refresh.

A retenir

- Materialise = cache SQL versionne.
- Documente la fraicheur attendue.

JSON dans SQL (idee)



JSON flexible, modele solide.

Beaucoup de moteurs stockent du **JSON** dans une colonne.

Usages

- Attributs flexibles (metadata produit).
- Eviter une explosion de colonnes rares.

Prudence

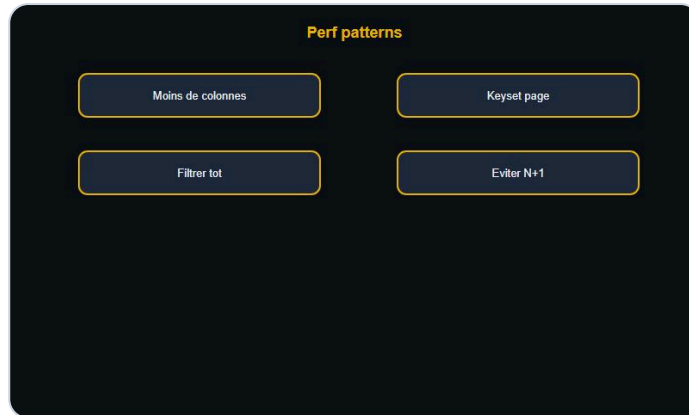
- Filtrer / indexer du JSON = plus complexe que des colonnes normales.
- Preferer colonnes classiques pour les champs **toujours** utilises.

```
-- Idee PostgreSQL
SELECT payload->>'ville' AS ville
FROM evenements;
```

A retenir

- JSON = flexibilite, pas remplacement du modele relationnel.

Patterns de performance



Moins de travail pour le moteur.

Checklist rapide

1. Sélectionner **moins de colonnes** (* coûte cher).
2. Filtrer **tot** (WHERE selectif).
3. Eviter N+1 applicatif (joindre ou batch).
4. Préférer **EXISTS** à **IN** avec sous-requete lourde selon cas.
5. Paginer avec cle stable (pas seulement **OFFSET** geant).

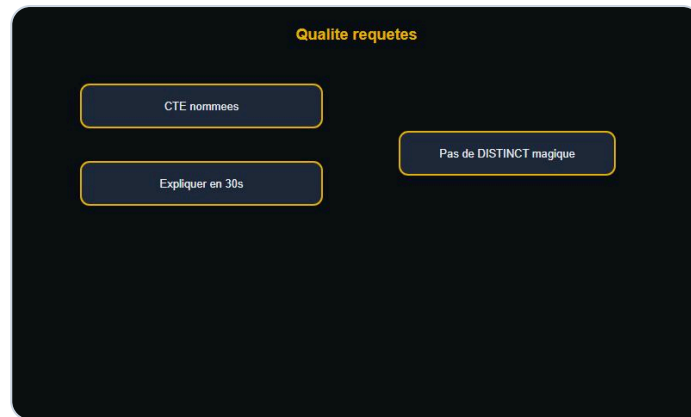
Pagination

```
-- Idee : keyset
SELECT * FROM commandes
WHERE id > :dernier_id
ORDER BY id
LIMIT 50;
```

A retenir

- Perf = moins de travail pour le moteur.
- Mesure avant d'optimiser.

Qualite des requetes



Lisibilite expert.

Style expert

- Alias courts mais lisibles (`c` , `cmd`).
- CTE nommees metier (`actifs` , `ca_mois`).
- Commentaires **pourquoi**, pas **quoi**.

Anti-patterns

- Sous-requetes correlees partout sans besoin.
- `DISTINCT` pour masquer un mauvais JOIN.
- Logique metier cachee dans 15 CASE imbriques.

> **Astuce DanielCraft** - Si tu ne peux pas expliquer la requete a voix haute en 30 s, simplifie.

A retenir

- Lisible aujourd'hui = maintenable demain.

Mini-projet : tableau de bord SQL



Dashboard SQL.

Objectif : produire un mini dashboard clients / commandes "niveau expert".

Livrables

1. CTE : CA par client sur 90 jours.
2. Fenetre : top 3 commandes par client (`ROW_NUMBER`).
3. EXPLAIN d'une requete lente, puis index propose.
4. Contrainte CHECK + FK documentees.

Jeu de donnees

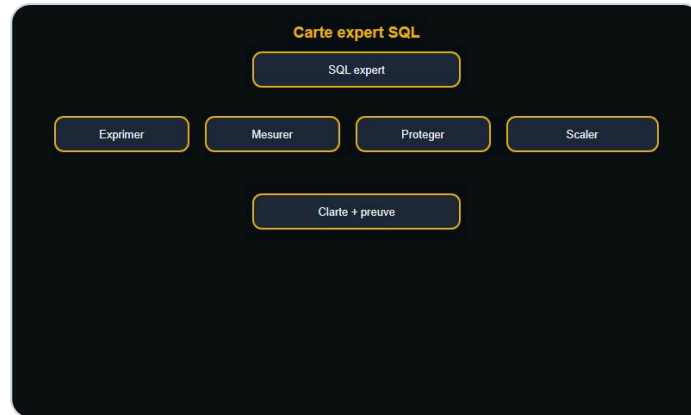
Tables `clients`, `commandes` (id, client_id, montant, date_cmd).

> **Astuce DanielCraft** - Verifie 3 chiffres a la main avant de presenter.

A retenir

- Mini-projet = fenetres + CTE + mesure + integrite.

Ce qu'il faut retenir



Carte expert.

Carte expert SQL

- **Exprimer** : fenêtrages, CTE, récursif.
- **Mesurer** : EXPLAIN, index cibles.
- **Protéger** : contraintes, transactions, isolation.
- **Scaler** : partitions, vues matérialisées (quand utile).

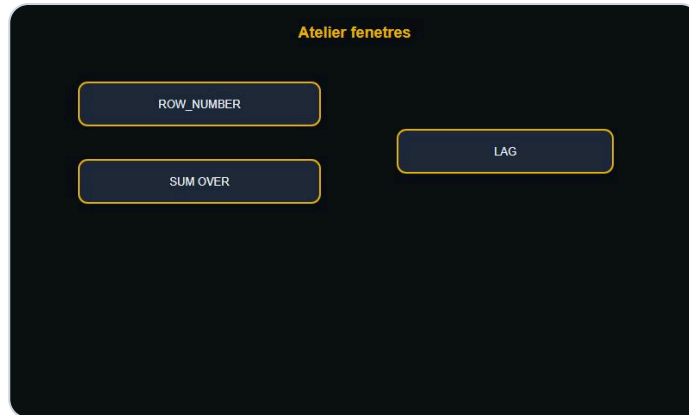
Mindset

- Clarte > ruse.
- Preuve (plan) > opinion.

A retenir

- Expert SQL = architecte de questions fiables.

Atelier : fenetres



Atelier fenetres.

Duree : 25 minutes.

Mission

Sur `commandes` :

1. Rang de chaque commande par `montant` **par client**.
2. Cumul des montants dans le temps par client (`SUM() OVER ... ORDER BY date`).
3. Ecart avec la commande precedente (`LAG`).

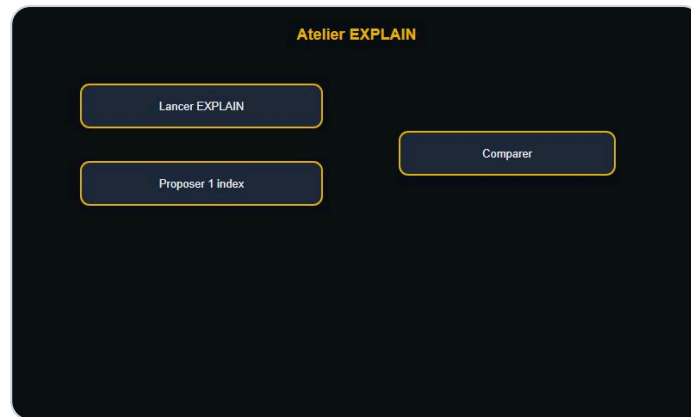
Criteres

- Une seule requete avec plusieurs colonnes fenetre OK.
- Verifier sur 1 client a la main.

A retenir

- `PARTITION BY` = groupe ; `ORDER BY` = ordre dans le groupe.

Atelier : EXPLAIN + index



Atelier EXPLAIN.

Duree : 20 minutes.

Mission

1. Ecrire `SELECT * FROM commandes WHERE client_id = ? AND date_cmd >= ?`.
2. Lancer EXPLAIN (ANALYZE si dispo).
3. Proposer **un** index composite.
4. Relancer EXPLAIN et comparer.

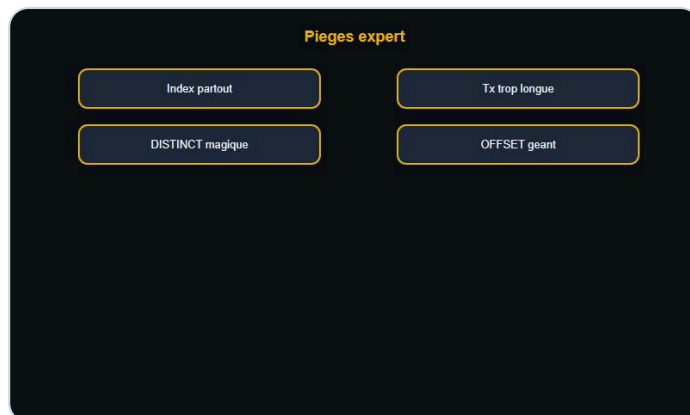
Questions

- Seq scan ou index scan ?
- Le filtre date est-il utilise efficacement ?

A retenir

- Un index, une hypothese, une mesure.

Erreurs classiques niveau expert



Pieges classiques.

Erreur	Pourquoi	Fix
Index partout	Ecritures lentes, plans pires	Indexer les filtres reels
DISTINCT pour "nettoyer"	Cache un JOIN mal fait	Corriger la jointure
Transaction geante	Verrous, timeouts	Decouper
OFFSET 100000	De plus en plus lent	Keyset pagination
Recursive sans limite	Explosion / cycle	Cap profondeur

> **Piege** - Optimiser une requete rare pendant que le N+1 de l'API detruit la prod.

A retenir

- Expert = eviter les fausses optimisations.

Bonnes pratiques SQL expert



Discipline prod.

1. **Francais d'abord** : ecrire l'intention, puis le SQL.
2. **CTE** pour les etapes.
3. **EXPLAIN** avant / apres changement.
4. **Contraintes** en base, pas seulement en app.
5. **Tests** : petit jeu de donnees + cas limites (NULL, doublons).
6. **Revue** : une deuxieme paire d'yeux sur les requetes critiques.

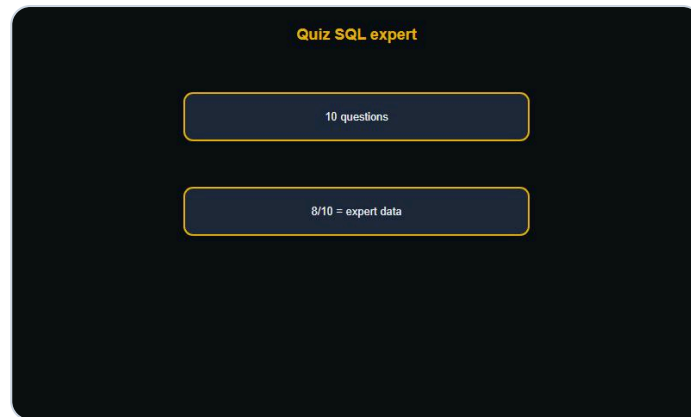
Checklist prod

- Index alignes sur EXPLAIN
- Pas de SELECT * en hot path
- Timeouts / limites de lignes
- Migrations reversible quand possible

A retenir

- Discipline > genie ponctuel.

Quiz SQL expert



Quiz SQL expert.

1. 1. Difference ROW_NUMBER et RANK ?
2. 2. A quoi sert une CTE ?
3. 3. Structure d'une CTE recursive ?
4. 4. Que regarder en premier dans EXPLAIN ?
5. 5. Ordre des colonnes dans un index composite ?
6. 6. Role d'une FOREIGN KEY ?
7. 7. Risque d'une transaction trop longue ?
8. 8. Quand partitionner ?
9. 9. Vue vs vue materialisee ?
10. 10. Pourquoi eviter OFFSET geant ?

Bareme

- 8/10+ : solide niveau expert.
- 6-7 : relire fenetres + EXPLAIN.
- Moins : reprendre intermediaire, puis revenir.

A retenir

- Le quiz valide la vision moteur + modele.

FINAL

Bravo !



Bravo. Tu mesures et tu structures.

Tu as terminé **SQL - Expert** : fenetres, CTE, plans, index, integrite, transactions, patterns prod.

Tu sais maintenant

- Ecrire des requetes lisibles et mesurees.
- Lire un plan d'execution.
- Proteger les donnees avec des contraintes.

Et maintenant ?

- Applique EXPLAIN sur une requete lente de ton projet.
- Remplace un OFFSET douloureux par une pagination keyset.
- Documente 3 contraintes manquantes.

> **DanielCraft** - Les donnees obeissent a qui pose les bonnes questions. Tu les poses.

A retenir

- Continue a mesurer. Le SQL expert se pratique.

Tu mesures. Tu structures. Tu livres.